Applied Cryptography, Project 2
Exploring the Security of One-Time and Many-Time Pads

Group 47
Natania Jeffery (nj2153)
Joe Raymond (cjr10071)
Shreyas Sankpal (sss10093)
Enoch Sanyaolu (mos7641)

Problem 1: Understanding the One-Time Pad

The one-time pad (OTP), also called the Vernam cipher, works on bits. The message *m*, the key *k* and the ciphertext *c* all have the same length *n*. Following the class notes:

Step	What it does
Key generation	KGen(1^n) picks *k* uniformly at random from $\{0,1\}^n$
Encryption	c = Enc(k, m) = m XOR k
Decryption	m = Dec(k, c) = c XOR k

Decryption works because XOR undoes itself. A bit XORed with itself gives 0, and XOR with 0 changes nothing:

$$c \text{ XOR } k = (m \text{ XOR } k) \text{ XOR } k = m \text{ XOR } (k \text{ XOR } k) = m \text{ XOR } 0 = m$$

Conditions

The class slides list three conditions. I added a fourth one, which the slides do not explicitly mention.

1. **The key is truly random.** Every key bit is picked independently with probability 1/2, so all 2^n keys are equally likely. A password, a line from a book, or the output of a predictable generator is not good enough, because then some keys are more likely than others and the attacker can use that.
2. **The key is at least as long as the message.** Every message bit needs its own key bit. If a shorter key is repeated, the pad turns into a Vigenère cipher on bits, and the repeating period can be found and broken the same way as in Project 1.
3. **The key is used only once, then destroyed.** If two messages use the same key, then c_1 XOR c_2 = m_1 XOR m_2. The key cancels out, and what is left is two English texts mixed together, which can be pulled apart. This is what Problems 3 and 4 are about.
4. **The key is secret and shared in advance.** Alice and Bob both need the same key before any message is sent, and it has to reach them over a channel Eva cannot read. The pad does nothing about this, it just assumes it.

Why it is perfectly secure

Shannon's definition of perfect secrecy says that the ciphertext tells the attacker nothing new about the plaintext:

$$\Pr[M = m \mid C = c] = \Pr[M = m] \qquad \text{for every message m and ciphertext c}$$

The class notes also give an equivalent form: any two messages are equally likely to produce a given ciphertext, where the probability is over the random key:

$$\Pr[\mathrm{Enc}(k, m_0) = c] = \Pr[\mathrm{Enc}(k, m_1) = c]$$

For the one-time pad this is easy to check. Fix any message *m* and any ciphertext *c* of *n* bits. Exactly one key turns *m* into *c*, namely k = m XOR c. The key is uniform, so

$$\Pr[\mathrm{Enc}(k, m) = c] = \Pr[k = m \text{ XOR } c] = 1/2^n$$

This number is the same for every *m*. So seeing *c* does not make any message more or less likely than before, and this holds even for an attacker with unlimited computing power. The only thing *c* gives away is the length. I also checked this in code in Problem 2: for Alice's ciphertext I computed a second key that decrypts the same ciphertext to a completely different message.

What it does not give

- It hides the content of the message but not its length.
- It gives no integrity. XOR is linear, so flipping a bit of *c* flips the same bit of *m* after decryption. If Eva knows *m*, she can turn *c* into an encryption of any other message m′ of the same length by XORing *c* with m XOR m′, without knowing the key.
- The key has to be as long as all the traffic that will ever be sent. Shannon showed this is not a flaw of this one cipher: any perfectly secret scheme needs at least as many keys as messages ($|K| \geq |M|$). This is why the OTP is rarely used and why key handling is where it breaks in practice.

How it is used in practice

1. Generate a long random pad and make exactly two copies.
2. Get one copy to each side by a secure route, for example in person or by courier.
3. For each message, use the next unused part of the pad. Both sides keep track of the position.
4. Destroy each part of the pad right after it is used, on both sides.

The Venona story from the handout shows what happens when step 4 fails. During World War II, under pressure from the German advance on Moscow, the Soviet factory that made the pads produced around 35,000 pages of duplicate key numbers, so some key material was used twice. The US Army's Signal Intelligence Service started the Venona project in 1943, and out of hundreds of thousands of intercepted messages, under 3,000 were partially or fully decrypted. So

the pad itself was never broken. The messages could be read only because some pages were used twice.

Problem 2: One-Time Pad Implementation

The code is in the notebook (proj2-Group47.ipynb, also in the appendix) and uses only the Python standard library. A few choices I made:

- **Bytes, not characters.** The message is encoded with UTF-8 and the XOR is done byte by byte, so any text works, not only A to Z.
- **Where the key comes from.** The key is made with secrets.token_bytes, which uses the operating system's random source. I did not use the random module. It is a Mersenne Twister, and after seeing enough of its output the rest can be predicted, which would break condition 1.
- **Hex files.** Most ciphertext and key bytes are not printable, so both are saved as hex, one line per file.
- **One shared helper.** xor_bytes(a, b) does the XOR. Alice and Bob both use it, and so do Problems 3 and 4.

Task 1: Alice's program

alice() asks for the message with input(), encodes it, makes a new random key of the same length, prints the ciphertext in hex, and saves the ciphertext and the key to ciphertext.txt and key.txt:

```
def otp_encrypt(plaintext):
    """Encrypt bytes with a fresh random key of the same length. Returns (ciphertext, key)."""
    key = secrets.token_bytes(len(plaintext))
    return xor_bytes(plaintext, key), key


def alice():
    """Alice's program: ask for a message, encrypt it, save the ciphertext and the key."""
    message = input("Message for Bob: ")
    ciphertext, key = otp_encrypt(message.encode("utf-8"))

    print("ciphertext (hex):", ciphertext.hex())

    save_hex(CIPHERTEXT_FILE, ciphertext)
    save_hex(KEY_FILE, key)
    print("saved", CIPHERTEXT_FILE, "and", KEY_FILE)
```

My run:

Item	Value
Message typed	Meet me at the library at 6 pm, bring the notes. (48 bytes)
ciphertext.txt	b523d70d87f45adc3773c70ce5dd25aa980d3086114e88f0a29a2fbdd8db3 79eafe2d210f7ccb1019fded3e285eba837

Item	Value
key.txt	f846b279a7993ffc5607e7788db805c6f16f42e76337a891d6ba199da8b61b becd90bb7e90ecc569fafebd8df18edb19

Both files were created. Each one holds 96 hex characters, which is 48 bytes, the same length as the message. The key is random, so running the cell again gives a different key and a different ciphertext for the same message. The values above are from my run.

Task 2: Bob's program

bob() only uses the two files, nothing from Alice's cell. It reads them back into bytes, checks that the key and the ciphertext have the same length (and raises an error if they do not), XORs them and decodes the result:

```
def otp_decrypt(ciphertext, key):
    """Recover the plaintext bytes. The key has to be exactly as long as the ciphertext."""
    if len(key) != len(ciphertext):
        raise ValueError("key and ciphertext have different lengths")
    return xor_bytes(ciphertext, key)


def bob():
    """Bob's program: read both files and print the plaintext."""
    ciphertext = load_hex(CIPHERTEXT_FILE)
    key = load_hex(KEY_FILE)
    print("plaintext:", otp_decrypt(ciphertext, key).decode("utf-8"))
```

Output:

```
plaintext: Meet me at the library at 6 pm, bring the notes.
```

This is exactly the message Alice typed.

Extra check: why Eva learns nothing

To connect this back to Problem 1, I took Alice's ciphertext and computed the key that would turn it into a different message of the same length, other key = ciphertext XOR other message:

Item	Value
Other message	Plans changed. Stay home, I will call you later.
Other key	e54fb663f4d439b4561da06981f305f9ec6c49a67921e5958eba6 69dafb25bf28f81b37c9becc86eeafebf83f18eda19
Ciphertext decrypted with the other key	Plans changed. Stay home, I will call you later.

Both keys are equally likely, so from the ciphertext alone Eva cannot tell which of the two messages was sent, or whether it was some other message of the same length.

Problem 3: Implementing Many-Time Pad

The only change from Problem 2 is that one key now encrypts all ten messages. The key is as long as the longest message (102 bytes), and message i is encrypted with the first len(m_i) bytes of it. The ten messages are quotes I like about security and programming:

#	Message	Bytes
1	The enemy knows the system.	27
2	Three may keep a secret, if two of them are dead.	49
3	Security is a process, not a product.	37
4	Talk is cheap. Show me the code.	32
5	Premature optimization is the root of all evil.	47
6	Simplicity is prerequisite for reliability.	43
7	The only truly secure system is one that is powered off.	56
8	Cryptography is typically bypassed, not penetrated.	51
9	Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin.	102
10	Any sufficiently advanced technology is indistinguishable from magic.	69

```python
def mtp_encrypt(plaintexts, key):
    """Encrypt every plaintext with the same key (the mistake this problem is about)."""
    if len(key) < max(len(p) for p in plaintexts):
        raise ValueError("key is shorter than the longest message")
    return [xor_bytes(p, key) for p in plaintexts]


def save_mtp(path, plaintexts, key, ciphertexts):
    """Save the key, then each plaintext and ciphertext, as labeled hex lines."""
    with open(path, "w") as f:
        f.write(f"key {key.hex()}\n")
        for i, (p, c) in enumerate(zip(plaintexts, ciphertexts), 1):
            f.write(f"m{i} {p.hex()}\n")
            f.write(f"c{i} {c.hex()}\n")
```

save_mtp writes everything to many_time_pad.txt, one labeled hex line per item: first the key, then m1, c1, m2, c2 and so on. The first three lines of my file:

Line	Hex
key	6f3ec11200fb6d33406edb999fea9ed4f3207d537508e09a180ce7e17c31dc37803d262 a447e4836062a42b8f052d594f15c64cffed664c7e0b76a4b1eefd017836e517c47ada01 2943632d838095e3d82bd795ac4959ffc828f3d907b3d430c78dd8b18636a
m1	54686520656e656d79206b6e6f777320746865207379737465 6d2e
c1	3b56a4326595085e394eb0f7f09dedf487481873067193ee7d61c9

The full file is printed in the appendix. As a check, the notebook reads the file back and decrypts every c line with the saved key. All ten messages come back correctly.

Patterns I found

The handout asks what can be seen just by looking at the outputs, and suggests changing the plaintexts or the key to confirm it. These are the patterns I found.

1. Same start, same ciphertext start. Each position always uses the same key byte, so the same character at the same position always gives the same ciphertext byte. The first six bytes of each ciphertext:

Ciphertext	First 6 bytes (hex)	Message starts with
c1	3b 56 a4 32 65 95	“The en”
c2	3b 56 b3 77 65 db	“Three ”
c3	3c 5b a2 67 72 92	“Securi”
c4	3b 5f ad 79 20 92	“Talk i”
c5	3f 4c a4 7f 61 8f	“Premat”
c6	3c 57 ac 62 6c 92	“Simpli”
c7	3b 56 a4 32 6f 95	“The on”
c8	2c 4c b8 62 74 94	“Crypto”
c9	2e 50 b8 7d 6e 9e	“Anyone”
c10	2e 50 b8 32 73 8e	“Any su”

c1 and c7 both start with “The ” and share their first four bytes. c2 (“Three”) shares two bytes with them, and c4 (“Talk”) only the first. c9 and c10 share “Any”, and c3 and c6 share “S”. With a new key for every message this could not happen. In Problem 4, eight of the eleven ciphertexts start with 71 fe for the same reason.

2. The key cancels out. For two ciphertexts made with the same key:

$$c_1 \text{ XOR } c_2 = (m_1 \text{ XOR } k) \text{ XOR } (m_2 \text{ XOR } k) = m_1 \text{ XOR } m_2$$

To check this, I encrypted the same ten messages again with a brand new key:

Value	Hex
c1 with the old key	3b56a4326595085e394eb0f7f09dedf487481873067193ee7d61c9
c1 with the new key	fa7aae3404c665547c5820ce21fe6905b971381c946aecd94cc5dd
c1 XOR c2, old key	00001745004e080c0000000b0a075341541b0043011c0758450448
c1 XOR c2, new key	00001745004e080c0000000b0a075341541b0043011c0758450448
m1 XOR m2	00001745004e080c0000000b0a075341541b0043011c0758450448

Every ciphertext changed, but c1 XOR c2 stayed exactly the same and equals m1 XOR m2. So once a key is reused, it does not matter what the key was.

3. Spaces show up as letters. This is Hint H. A space is 0x20 = 0010 0000. Uppercase letters start with 010 and lowercase letters with 011. So:

- letter XOR space: only the 0x20 bit flips, so the result is the same letter in the other case. For example, ‘a’ XOR ‘ ’ = ‘A’.
- letter XOR letter: 010 XOR 010 and 011 XOR 011 give 000, and 010 XOR 011 gives 001. The result is below 0x40, so it is never a letter.

Below is c1 XOR c7 with one character per box. The XOR row shows the letter when the byte is a letter, 0 when it is zero, and a dot otherwise:

m1	T	h	e		e	n	e	m	y		k	n	o	w	s		t	h	e		s	y	s	t	e	m	.
m7	T	h	e		o	n	l	y		t	r	u	l	y		s	e	c	u	r	e		s	y	s	t	e
XOR	0	0	0	0	.	0	.	.	Y	T	.	.	.	.	S	S	.	.	.	R	.	Y	0	.	.	.	K

Every letter in the XOR row sits under a space in one of the two messages, and it is the other message’s character with the case flipped. The K at the end is the exception. It comes from the period: ‘.’ XOR ‘e’ = 0x2E XOR 0x65 = 0x4B = ‘K’. Punctuation also starts with 0010, so it can fake a space. So a letter in the XOR means “very likely a space”, not “always a space”.

4. The rule holds over all pairs. Over all 45 pairs of messages and every position:

Characters at that position	XOR of the two ciphertexts was a letter
space and letter	436 of 436 times
letter and letter	0 of 1233 times

So from the ciphertexts alone, a letter in c_i XOR c_j points to a space in one of the two messages.

5. One known message opens the rest. From k = c XOR m (Hint F), knowing one plaintext gives the key bytes at those positions. I assumed Eva knows only message 8 (51 bytes). Using c_8 XOR m_8 as a partial key, every other message decrypted correctly for its first 51 characters, for example “The only truly secure system is one that is powered” and “Anyone who considers arithmetical methods of produc”.

All five of these get used in Problem 4.

Problem 4: Cryptanalysis of Many-Time Pad

Task 1: Attack strategy

Eva has ten ciphertexts and a target, all encrypted with the same key, and she knows the plaintexts are English. She wants the target. My plan:

1. **Look first.** Compare the ciphertexts column by column and XOR some pairs. The key cancels (Hint G), so equal bytes and zero bytes show where messages have the same characters.

2. **Space trick (Hint H).** For each position, find the ciphertext that gives a letter when XORed with most of the others. That one probably has a space there, so the key byte is that ciphertext byte XOR 0x20. This part runs by itself and gives a first partial key.
3. **Crib dragging (Hint J).** Slide a likely word, such as " the ", along the target. At each position, work out the key bytes that word would need and check whether the other ten messages turn into text there.
4. **Human in the loop (Hints F and J).** Decrypt all eleven ciphertexts with the partial key, read them, and guess missing letters, words or whole sentences. Each guess becomes key bytes through k = c XOR m.
5. **Verify.** Keep a guess only if all the other messages still read as English at the same positions. A wrong guess turns the other messages into garbage right away, so this check is strong.
6. **Decrypt the target** with the final key.

The ciphertexts are 78, 113, 90, 90, 103, 98, 110, 46, 123, 52 bytes long and the target is 59 bytes. Every position of the target is covered by at least 8 of the other ciphertexts, so each guess about the target can be checked against many messages.

Task 2: Implementation

The notebook does this with a few small functions:

Function	What it does
guess_key_from_spaces(cts)	Step 2. For every position and every ciphertext, counts how many other ciphertexts give a letter or zero when XORed with it. If the best one has votes from at least half of the others, the key byte is set to that ciphertext byte XOR 0x20. Otherwise it stays unknown (None).
decrypt_partial(c, key) and show_all(key)	Decrypt with a partly known key. An unknown key byte prints as _ and a non-printable result as #. show_all prints all eleven messages so I can read them side by side.
crib_drag(ct, others, word)	Step 3. Tries the word at every position of ct and scores each position by how many other messages become text (letters, digits, space, comma, period, question mark).
apply_crib(key, ct, position, guess)	Step 4. Assumes ct decrypts to guess at that position and sets those key bytes to c XOR m.
compare(auto, final, positions)	At the end, counts how many key bytes the automatic step got right, got wrong or left empty.

The two main functions:

```
def guess_key_from_spaces(cts):
    """Guess key bytes from likely spaces. Positions without enough evidence stay None."""
    length = max(len(c) for c in cts)
```

```
    key = [None] * length
    for p in range(length):
        column = [c[p] for c in cts if len(c) > p]
        best_votes, best_byte = 0, None
        for i, ci in enumerate(column):
            # vote: this ciphertext XOR another one gives a letter, or zero
            votes = sum(1 for j, cj in enumerate(column)
                        if j != i and (is_letter(ci ^ cj) or ci == cj))
            if votes > best_votes:
                best_votes, best_byte = votes, ci ^ 0x20
        if best_votes >= max(2, (len(column) - 1) // 2):
            key[p] = best_byte
    return key
```

```
def apply_crib(key, ct, position, guess):
    """Set key bytes so that ct decrypts to guess at this position (k = c XOR m)."""
    for offset, ch in enumerate(guess.encode("utf-8")):
        key[position + offset] = ct[position + offset] ^ ch
```

The only manual part is choosing the cribs. I typed each one into a cell after reading the output of the previous one. Everything else is code, and the whole notebook runs from top to bottom without errors.

Task 3: Results

Step 1: raw ciphertexts. Eight of the eleven ciphertexts start with 71 fe, so those eight messages begin with the same two characters. c3 starts with 72 fe, so it differs from them only in the first character. c1 XOR c7 starts with twelve zero bytes:

```
00 00 00 00 00 00 00 00 00 00 00 00 0d 0a 1a 50 3c 20 45 1d
```

so messages 1 and 7 share their first twelve characters. All of this is visible without knowing anything about the key.

Step 2: space trick. The automatic step set 88 of the 123 key bytes. Decrypting with it gave the following (_ is an unknown key byte, # is a non-printable result):

	Decryption with the automatic key
c1	y was the n__ IT off__ial hospitalize_? H_ ac_id__tal_y _oucwe_ the firzwall
c2	y are the i__urance __d premiums for _ll _pp _ev__ope_s _norro_sly high Because they f_e __wa__ c__s_i_g d____
c3	# ere are 10 __pes of __ople in the wor_d. _hos_ w__ un_er_tan{ _inary an{ those who don _
c4	6; for punish__nt, whe__ are naughty di_k d_ive_ s__t? _he_ arz _lways seqt to a Boot caj_
c5	at did the __mputer __ on its much-aw_ite_ va_at__n a_ t_e bza_h? It ha{ a great time t_rf__g __e __t
c6	?'w many comp__er prog__mmers does it t_ke _o c_an__ a _ig_t bjl_? None; khat's a hardwau_ p__bl__

	Decryption with the automatic key
c7	y was the c__puter s__entist angry wh_n t_e s_ud__t c_ac_ed ~ _ame compjter joke? He dn_ n__ l__e __ _n_ bi_
c8	at does a b__y compu__r call his fath_r?D_ta
c9	y do cats l__e sitti__ in front of th_ co_put_r __l d_y _ong _ecause twey don't want s_ l__ t__ m__s_ _ut _____
c10	at kind of __ney do __mputer scientis_s u_e? _ac__
T	en using a __ream ci__er never use th_ ke_ mo_e __an _nc_

Most of it is already readable, and the messages are clearly computer jokes. The first two characters are wrong in every message. Almost all messages begin with the same two letters, so the zero XORs counted as votes for a space. Most of the blanks are columns where no message has a space, so the trick has nothing to work with there.

Step 3: crib dragging. I slid " the " along the target. The two best positions:

Position	Score	What the other messages show there
36	9 of 10	'ized?', 'or al', 'world', ' disk', '-awai', 't tak', ' when', 'ather', ' the ', 'tists'
12	8 of 10	'nbk7\x0c', 'iiob7', ' seg ', 'hjyy1', ' dsz5', 'prhr7', 'chqg0', 'bf~ne', 'lhjre', ' jsy '

Position 36 has real word pieces ('ized?', 'world', ' disk', ' when') and matches "use th_ ke_" in the target. Position 12 scores almost as high, but its pieces are junk like 'iiob7' and 'chqg0' that only pass because my filter accepts any letter or digit. So the score narrows things down, but I still had to read the pieces to choose. The one miss at position 36, '-awai', has a hyphen, which my filter did not allow. It is part of "much-awaited" in message 5.

Step 4: guessing the target. The partial target, " en using a __ream ci__er never use th_ ke_ mo_e __an _nc_", reads like the well-known warning "When using a stream cipher never use the key more than once". That sentence is 59 characters and the target is 59 bytes. I applied it as a crib, which set key bytes 0 to 58, and decrypted all eleven again:

	Decryption after the target crib
c1	Why was the new IT official hospitalized? He accidentally toucwe_ the firzwall
c2	Why are the insurance and premiums for all app developers enorro_sly high Because they f_e __wa__ c__s_i_g d____
c3	There are 10 types of people in the world. Those who understan{ _inary an{ those who don _
c4	As for punishment, where are naughty disk drives sent? They arz _lways seqt to a Boot caj_
c5	What did the computer do on its much-awaited vacation at the bza_h? It ha{ a great time t_rf__g __e __t

	Decryption after the target crib
c6	How many computer programmers does it take to change a light bjl_? None; khat's a hardwau_ p__bl__
c7	Why was the computer scientist angry when the student cracked ~ _ame compjter joke? He dn_ n__ l__e __ _n_ bi_
c8	What does a baby computer call his father?Data
c9	Why do cats love sitting in front of the computer all day long _ecause twey don't want s_ l__ t__ m__s_ _ut _____
c10	What kind of money do computer scientists use? Cache
T	When using a stream cipher never use the key more than once

Every message now reads as normal English up to position 58, including the parts that were blank or wrong before: the “Wh” at the start, “hospitalized?”, “disk drives”, “much-awaited vacation” and “love sitting”. Ten other messages confirm the guess, so the target is decrypted. (After position 58 the lines still use the automatic key, which is why they have errors there.)

Step 5: the rest of the key. This is not needed for the target, but finishing the other messages is a good extra check. I guessed the full text of each joke from what was visible, a few at a time, and read all messages again after each round:

Round	Cribs (full messages)	What it revealed in the other messages
1	Messages 1, 3 and 6	Message 2: “…they are always c__s_i_g d____” Message 5: “…surfing the __t” Message 7: “…He did not like __ _n_ bi_” Message 9: “…let the m__s_ _ut”
2	Messages 5 and 7	Message 2: “…crashing do___” Message 9: “…mouse out o___”
3	Message 9	Message 2: “…crashing down!”, which confirms the guess for message 9 at positions 110 to 112

No guess broke any other message. The last ten key bytes (positions 113 to 122) only appear in message 9, so nothing else can confirm them. They are right only if “their sight” is right. This does not affect the target.

Final result

Target message: When using a stream cipher never use the key more than once

All ten messages:

#	Plaintext
1	Why was the new IT official hospitalized? He accidentally touched the firewall

#	Plaintext
2	Why are the insurance and premiums for all app developers enormously high? Because they are always crashing down!
3	There are 10 types of people in the world. Those who understand binary and those who don't
4	As for punishment, where are naughty disk drives sent? They are always sent to a Boot camp
5	What did the computer do on its much-awaited vacation at the beach? It had a great time surfing the net
6	How many computer programmers does it take to change a light bulb? None; that's a hardware problem
7	Why was the computer scientist angry when the student cracked a lame computer joke? He did not like it one bit
8	What does a baby computer call his father?Data
9	Why do cats love sitting in front of the computer all day long? Because they don't want to let the mouse out of their sight
10	What kind of money do computer scientists use? Cache

Recovered key (123 bytes, hex):

269663ee34e87b52127919f7a7e936a2cc4f1aa095d6ef80915ef8f6836ab7caecac80967d8b78780476c793ef94a3ff34038a32e810db82caf42e3e21c87fdd5a276881bdb8ecb2389ba827c42cfa60da288b9261af54d2adbd442c8eb76d9441e4391a2c7e7211a219a2e432fa3ae6d2328ed0776b9fb33bfa0c

Analysis

How much the automatic step did on its own:

Key positions	Right	Wrong	Empty
Target (0 to 58)	46	2	11
Whole key (0 to 122)	83	5	35

- For the target, the space trick alone got 46 of 59 key bytes right (78%). That was enough to read most words, and one human guess did the rest.
- Of the 40 key bytes it got wrong or left empty, 38 are in columns where no message has a space. The other two are near the end, where only one or two ciphertexts are left. So the trick only works in columns that have a space and enough other messages to vote.
- The wrong bytes at positions 0 and 1 come from counting zero XORs as votes. That helps when two messages have a space in the same column, but at the start almost every message has “Wh”, so the zeros outvoted everything else. The wrong bytes at 62, 73 and 88 are in columns with a question mark or an apostrophe. Punctuation starts with 0010

like the space, so it can also give a letter when XORed with a letter, the same way the period gave the K in Problem 3.

- The handout says only space, comma, period and question mark are used, but the plaintexts also contain ! ' - and ; (in messages 2, 3, 5, 6, 9). This did not stop the attack, but a strict character filter rejects correct guesses, as it did with '-awai' in Step 3. In practice the filter has to be loose and a human has to read the output.
- The manual part was only a few guesses, but without it the attack would have stopped at a partly readable target. One sentence-level guess on the target fixed all of it at once. What made the guesses safe is the cross-check: with ten other messages, a wrong guess shows up as garbage immediately.

What this says about key reuse:

- Reusing the key even once already leaks m_1 XOR m_2. Here the attack never needed the key: XOR removed it, and the structure of English (spaces, common words, predictable endings) did the rest.
- Each extra message under the same key makes the attack easier. It adds another vote in the space trick and another check on every guess.
- This is not specific to the one-time pad. Any stream cipher fails the same way if its keystream repeats. WEP is a known example: its 24-bit IV repeats, so RC4 keystreams get reused. AES in CTR or GCM mode has the same problem if a nonce is reused with the same key. The target message says exactly this.

References

1. Assignment handout, "Exploring the Security of One-Time and Many-Time Pads".
2. Class slides (Overview of Primitives, Overview of Security) and the class notes on the one-time pad.
3. Chapter 2 reading, Theorem 2.10: the one-time pad is perfectly secret if the key is random and used only once.
4. Venona project, Wikipedia: https://en.wikipedia.org/wiki/Venona_project
5. D. Boneh, CS 255 Homework 1: http://crypto.stanford.edu/~dabo/cs255/hw_and_proj/hw1.html

Appendix: Jupyter Notebook

Contents of proj2-Group47.ipynb with its output, included so this file covers both parts of the submission. The notebook is submitted separately as well.

Applied Cryptography, Project 2

Group 47

Natania Jeffery (nj2153)

Joe Raymond (cjr10071)

Shreyas Sankpal (sss10093)

Enoch Sanyaolu (mos7641)

Code for Problems 2, 3 and 4. Problem 1 is theory only, so it is in the report.

Only the standard library is needed. Keys come from `secrets`, because `random` is predictable and not meant for keys.

In [1]:
```
import secrets
from itertools import combinations


def xor_bytes(a, b):
    """XOR two byte strings position by position. Stops at the shorter one."""
    return bytes(x ^ y for x, y in zip(a, b))


def is_letter(b):
    """True if the byte is an ASCII letter, A-Z or a-z."""
    return 65 <= b <= 90 or 97 <= b <= 122
```

Problem 2: One-time pad between Alice and Bob

Alice and Bob only share two files, one for the ciphertext and one for the key. Both are saved as hex so they are normal text files. A new key is made for every message, with the same length as the message.

In [2]:
```
KEY_FILE = "key.txt"
CIPHERTEXT_FILE = "ciphertext.txt"


def save_hex(path, data):
    """Write bytes to a text file as one line of hex."""
    with open(path, "w") as f:
        f.write(data.hex())


def otp_encrypt(plaintext):
    """Encrypt bytes with a fresh random key of the same length. Returns (ciphertext, key)."""
    key = secrets.token_bytes(len(plaintext))
    return xor_bytes(plaintext, key), key
```

```
def alice():
    """Alice's program: ask for a message, encrypt it, save the ciphertext and the key."""
    message = input("Message for Bob: ")
    ciphertext, key = otp_encrypt(message.encode("utf-8"))

    print("ciphertext (hex):", ciphertext.hex())

    save_hex(CIPHERTEXT_FILE, ciphertext)
    save_hex(KEY_FILE, key)
    print("saved", CIPHERTEXT_FILE, "and", KEY_FILE)


alice()
```

```
Message for Bob: Meet me at the library at 6 pm, bring the notes.
ciphertext (hex):
b523d70d87f45adc3773c70ce5dd25aa980d3086114e88f0a29a2fbdd8db379eafe2d210f7ccb1019fded3e285eba837
saved ciphertext.txt and key.txt
```

In [3]:
```
# What Alice left on disk
for path in (CIPHERTEXT_FILE, KEY_FILE):
    with open(path) as f:
        print(path, "->", f.read())
```

```
ciphertext.txt ->
b523d70d87f45adc3773c70ce5dd25aa980d3086114e88f0a29a2fbdd8db379eafe2d210f7ccb1019fded3e285eba837
key.txt ->
f846b279a7993ffc5607e7788db805c6f16f42e76337a891d6ba199da8b61bbecd90bb7e90ecc569fafebd8df18edb19
```

In [4]:
```
def load_hex(path):
    """Read a hex file written by save_hex and return the bytes."""
    with open(path) as f:
        return bytes.fromhex(f.read().strip())


def otp_decrypt(ciphertext, key):
    """Recover the plaintext bytes. The key has to be exactly as long as the ciphertext."""
    if len(key) != len(ciphertext):
        raise ValueError("key and ciphertext have different lengths")
    return xor_bytes(ciphertext, key)


def bob():
    """Bob's program: read both files and print the plaintext."""
    ciphertext = load_hex(CIPHERTEXT_FILE)
    key = load_hex(KEY_FILE)
    print("plaintext:", otp_decrypt(ciphertext, key).decode("utf-8"))


bob()
```

```
plaintext: Meet me at the library at 6 pm, bring the notes.
```

One more check, which ties back to Problem 1. For any other message with the same length there is a key that turns it into exactly this ciphertext. So Eva, holding only the ciphertext, has no way to tell which message was sent.

In [5]:
```
ciphertext = load_hex(CIPHERTEXT_FILE)
other = "Plans changed. Stay home, I will call you later."
other = other.ljust(len(ciphertext))[:len(ciphertext)]   # same length as Alice's message

other_key = xor_bytes(ciphertext, other.encode("utf-8"))
```

```
print("other key  :", other_key.hex())
print("decrypts to:", otp_decrypt(ciphertext, other_key).decode("utf-8"))
```

```
other key  :
e54fb663f4d439b4561da06981f305f9ec6c49a67921e5958eba669dafb25bf28f81b37c9becc86eeafebf83f18eda19
decrypts to: Plans changed. Stay home, I will call you later.
```

Problem 3: Many-time pad

Same XOR as before, but now one key is used for all ten messages. The key is as long as the longest message, and each message uses the first len(message) bytes of it. The messages are some of my favorite quotes about security and programming.

In [6]:
```
MTP_FILE = "many_time_pad.txt"

messages = [
    "The enemy knows the system.",
    "Three may keep a secret, if two of them are dead.",
    "Security is a process, not a product.",
    "Talk is cheap. Show me the code.",
    "Premature optimization is the root of all evil.",
    "Simplicity is prerequisite for reliability.",
    "The only truly secure system is one that is powered off.",
    "Cryptography is typically bypassed, not penetrated.",
    "Anyone who considers arithmetical methods of producing random digits is, of course, in a
state of sin.",
    "Any sufficiently advanced technology is indistinguishable from magic.",
]
plaintexts = [m.encode("utf-8") for m in messages]


def mtp_encrypt(plaintexts, key):
    """Encrypt every plaintext with the same key (the mistake this problem is about)."""
    if len(key) < max(len(p) for p in plaintexts):
        raise ValueError("key is shorter than the longest message")
    return [xor_bytes(p, key) for p in plaintexts]


def save_mtp(path, plaintexts, key, ciphertexts):
    """Save the key, then each plaintext and ciphertext, as labeled hex lines."""
    with open(path, "w") as f:
        f.write(f"key {key.hex()}\n")
        for i, (p, c) in enumerate(zip(plaintexts, ciphertexts), 1):
            f.write(f"m{i} {p.hex()}\n")
            f.write(f"c{i} {c.hex()}\n")


mtp_key = secrets.token_bytes(max(len(p) for p in plaintexts))
mtp_cts = mtp_encrypt(plaintexts, mtp_key)
save_mtp(MTP_FILE, plaintexts, mtp_key, mtp_cts)

print("key length:", len(mtp_key), "bytes, saved", MTP_FILE)
```

```
key length: 102 bytes, saved many_time_pad.txt
```

In [7]:
```
with open(MTP_FILE) as f:
    print(f.read())
```

```
key
6f3ec11200fb6d33406edb999fea9ed4f3207d537508e09a180ce7e17c31dc37803d262a447e4836062a42b8f052d594f1
```

```
5c64cffed664c7e0b76a4b1eefd017836e517c47ada012943632d838095e3d82bd795ac4959ffc828f3d907b3d430c78dd
8b18636a
m1 54686520656e656d79206b6e6f777320746865207379737 4656d2e
c1 3b56a4326595085e394eb0f7f09dedf487481873067193ee7d61c9
m2
5468726565206d6179206b6565702061207365637265742c2069662074776f206f66207468656d2061726520646561642e
c2
3b56b37765db0052394eb0fcfa9abeb5d3531830076d94b6386581c10846b317ef5b065e2c1b2516675827989437b4f0df
m3 536563757269747920697320612070726f636573732c206e6f7420612070726f647563742e
c3 3c5ba2677292194a6007a8b9fecaeea69c4318200624c0f47778c7805c41ae58e448455e6a
m4 54616c6b206973206368656170 2e2053686f77206d652074686520636f64652e
c4 3b5fad7920921e132306bef8efc4be879b4f0a73186dc0ee7069c7821355b919
m5 5072656d6174757265206f7074696d697a6174696f6e206973207468652072 6f6f74206f6620616c6c206576696c2e
c5 3f4ca47f618f1841254eb4e9eb83f3bd8941093a1a66c0f36b2c93891911ae58ef490645225e295a6a0a27ce993efb
m6 53696d706c6963697479206973207072657265717569369746520666f722072656c696162696c6974792e
c6 3c57ac626c920e5a3417fbf0eccaeea696521822006193f36c69c7871343fc45e5514f4b2617245f72536c
m7
546865206f6e6c79207472756c792073656375726520737973746 56d206973206f6e65207468617420697320706f7765726564206f66662e
c7
3b56a4326f95014a601aa9ecf393bea796430821102893e36b78828c5c58af17ef53430a3016294226433198803da2f1833900ef91b002e9
m8
43727970746f6772617068792069732074797069636 16c6c792062797061737365642c206e6f742070656e6574726174656 42e
c8
2c4cb86274940a41211eb3e0bf83edf487590d3a16698cf6612c85980c50af44e5590a0a2a113c16764f2cdd8420b4e094384a
m9
416e796f6e652077686f20636f6e7369646572732061726974686d6574696361 6c206d6574686f6473206f662070726f647563696e672072616e646f6d206469676974732069732c206f6620636f757273652c20696e20612073746174 65206f662073696e2e
c9
2e50b87d6e9e4d442801fbfaf084edbd97450f20556992f36c648a840858bf56ec1d4b4f30162752750a2dded022a7fb952907a690b144b581d90e2473cfb47ee407250f67c4d33eb45954f85b662b4ff1d8557aadfbbf9da2fc49f10f5863631efdf8710d44
m10
416e792073756666696369656e746c7920616476616e6365642074656 3686e6f6c6f677920697320696e646973746 96e677569736861626c652066726f6d206d616769632e
c10
2e50b832738e0b55290db2fcf19ef2add3411925146683ff7c2c93841f59b258ec52415364173b166f4426d18326bcfa96290dbc96b706ab85970c397182f07ae209381f69
```

In [8]:
```python
# Read the file back and decrypt each c line with the saved key.
with open(MTP_FILE) as f:
    saved = dict(line.split() for line in f)

saved_key = bytes.fromhex(saved["key"])
ok = all(
    xor_bytes(bytes.fromhex(saved[f"c{i}"]), saved_key) == bytes.fromhex(saved[f"m{i}"])
    for i in range(1, len(messages) + 1)
)
print("all messages decrypt correctly:", ok)
```

```
all messages decrypt correctly: True
```

Looking for patterns

1. Same start, same ciphertext start. Every column uses the same key byte, so the same character in the same position always gives the same ciphertext byte.

In [9]:
```python
for i, c in enumerate(mtp_cts, 1):
```

```
    print(f"c{i:<3}{c[:6].hex(' ')}   {messages[i - 1][:6]!r}")
```

```
c1  3b 56 a4 32 65 95   'The en'
c2  3b 56 b3 77 65 db   'Three '
c3  3c 5b a2 67 72 92   'Securi'
c4  3b 5f ad 79 20 92   'Talk i'
c5  3f 4c a4 7f 61 8f   'Premat'
c6  3c 57 ac 62 6c 92   'Simpli'
c7  3b 56 a4 32 6f 95   'The on'
c8  2c 4c b8 62 74 94   'Crypto'
c9  2e 50 b8 7d 6e 9e   'Anyone'
c10 2e 50 b8 32 73 8e   'Any su'
```

c1 and c7 both start with "The " and share 4 bytes. c2 ("Three") shares the first 2 with them, and c4 ("Talk") only the T. c9 and c10 share "Any", and c3 and c6 share "S". With a fresh key for every message this would not happen.

2. The key cancels. c1 XOR c2 = (m1 XOR k) XOR (m2 XOR k) = m1 XOR m2. To check it, I encrypted the same messages again with a completely new key.

In [10]:
```
new_key = secrets.token_bytes(len(mtp_key))
new_cts = mtp_encrypt(plaintexts, new_key)

print("c1, old key:", mtp_cts[0].hex())
print("c1, new key:", new_cts[0].hex())
print()
print("c1 XOR c2, old key:", xor_bytes(mtp_cts[0], mtp_cts[1]).hex())
print("c1 XOR c2, new key:", xor_bytes(new_cts[0], new_cts[1]).hex())
print("m1 XOR m2         :", xor_bytes(plaintexts[0], plaintexts[1]).hex())
```

```
c1, old key: 3b56a4326595085e394eb0f7f09dedf487481873067193ee7d61c9
c1, new key: fa7aae3404c665547c5820ce21fe6905b971381c946aecd94cc5dd

c1 XOR c2, old key: 00001745004e080c0000000b0a075341541b0043011c0758450448
c1 XOR c2, new key: 00001745004e080c0000000b0a075341541b0043011c0758450448
m1 XOR m2         : 00001745004e080c0000000b0a075341541b0043011c0758450448
```

The ciphertexts changed completely, but c1 XOR c2 did not, and it equals m1 XOR m2. A new key does not help once it is reused.

3. Spaces show up as letters. A space is 0x20 (0010 0000). XOR with a letter only flips the case bit, so 'a' XOR ' ' = 'A'. Letters start with 010 or 011, so two letters XOR to something starting with 000 or 001, which is below 0x40 and never a letter. Below is c1 XOR c7, printed as the letter when the byte is a letter, 0 when it is zero, and . otherwise.

In [11]:
```
def xor_view(a, b):
    """Show a XOR b as text: the letter if it is a letter, 0 if zero, '.' otherwise."""
    out = ""
    for x in xor_bytes(a, b):
        out += "0" if x == 0 else chr(x) if is_letter(x) else "."
    return out


n = len(messages[0])
print("m1 :", messages[0])
print("m7 :", messages[6][:n])
print("XOR:", xor_view(mtp_cts[0], mtp_cts[6]))
```

```
m1 : The enemy knows the system.
m7 : The only truly secure syste
```

```
XOR: 0000.0..YT....SS...R.Y0...K
```

Each letter sits where one of the two messages has a space, and it is the other message's character with the case flipped. The K at the end is different: it comes from the period. Punctuation also starts with 0010, so it can fake a space. That makes the rule "very likely", not "always".

4. Checking the rule on all pairs. I went over every pair of messages and every position and looked at the XOR of the two ciphertexts.

In [12]:
```python
# for each case: [times the XOR was a letter, total]
counts = {"space + letter": [0, 0], "letter + letter": [0, 0]}

for i, j in combinations(range(len(mtp_cts)), 2):
    for p, x in enumerate(xor_bytes(mtp_cts[i], mtp_cts[j])):
        a, b = plaintexts[i][p], plaintexts[j][p]
        if is_letter(a) and is_letter(b):
            kind = "letter + letter"
        elif (a == 0x20 and is_letter(b)) or (b == 0x20 and is_letter(a)):
            kind = "space + letter"
        else:
            continue
        counts[kind][0] += is_letter(x)
        counts[kind][1] += 1

for kind, (hits, total) in counts.items():
    print(f"{kind}: XOR is a letter in {hits} of {total} cases")
```

```
space + letter: XOR is a letter in 436 of 436 cases
letter + letter: XOR is a letter in 0 of 1233 cases
```

So looking only at ciphertexts, a letter in ci XOR cj means one of the two messages has a space there. This is the main tool for Problem 4.

5. One known message opens the rest. If Eva knows or guesses one plaintext, she gets the key for those positions from k = c XOR m (Hint F), and then she can read every other message at the same positions. Here I assume she knows only message 8.

In [13]:
```python
known = plaintexts[7]
partial_key = xor_bytes(mtp_cts[7], known)   # key bytes 0 .. len(known) - 1

for i, c in enumerate(mtp_cts, 1):
    print(f"m{i:<3}{xor_bytes(c, partial_key).decode('utf-8')}")
```

```
m1  The enemy knows the system.
m2  Three may keep a secret, if two of them are dead.
m3  Security is a process, not a product.
m4  Talk is cheap. Show me the code.
m5  Premature optimization is the root of all evil.
m6  Simplicity is prerequisite for reliability.
m7  The only truly secure system is one that is powered
m8  Cryptography is typically bypassed, not penetrated.
m9  Anyone who considers arithmetical methods of produc
m10 Any sufficiently advanced technology is indistingui
```

Every message is readable up to the length of message 8. These five observations are what the attack in Problem 4 is built on: XOR to remove the key, spaces to get the first key bytes, and guessed text to get the rest.

Problem 4: Breaking the many-time pad

Eva has 10 ciphertexts and a target, all made with the same key, and she knows the plaintexts are English.

My plan:

1. Look at the raw ciphertexts and at XORs of pairs, where the key cancels (Hint G).
2. Guess key bytes automatically with the space trick from Problem 3 (Hint H).
3. Use crib dragging to test likely words (Hint J).
4. Read the partial plaintexts, guess the missing text, and turn each guess into key bytes with k = c XOR m (Hint F).
5. Keep a guess only if all the other messages still read as English at the same positions.

In [14]:
```
# Copied from the assignment (long hex strings split into shorter lines)
CIPHERTEXTS_HEX = [
    # 1
    "71fe1ace4389087266117cd7c98c4182851b3acff3b086e3f83f94d6eb05c4ba"
    "85d8e1fa14f11d1c3b568ff6cff5c09c5d67ef5c9c71b7eeb3d45a5154ab17b8"
    "3e071ce9d8988adb4afedf46a840",
    # 2
    "71fe1ace559a1e7266117cd7ce8745d7be2e74c3f0f68eeef57e8884e607debf"
    "81dfa0f012f95819681ae7f29fe4839b5175ef5e8760bef0b9d44b504eba12b2"
    "2f5404f89dd085d550a48865a14f9b15a94dabe609ca2df2cccf210cefdb1af5"
    "389719795e1f0179cb77c5c456954d88f3",
    # 3
    "72fe069c51c81a20775928c7879d4fd2a93c3acff3f69fe5fe2e9493a303d9ea"
    "98c4e5b60ae40a146058e7c787fbd09a1474e25dc865b5e6af865d4a40a61bfd"
    "384e06e0cfc1ccd356ff8853ac438905fa5fe3fd41cb3bbc8ac9",
    # 4
    "67e543885b9a5b2267177084cf8453ccb8633ad7fdb39de5b13f8a93a304d6bf"
    "8bc4f4ef5def110b6f56a3e186e2c68c1470ef5c9c2ffbd6a291571e40ba1afd"
    "3b4b1fe0c4cbccc15df5dc07b043da01fa6ae4fd158f37b3c0cd",
    # 5
    "71fe029a148c1236320d7192878a59cfbc3a6ec5e7f68befb13196d6ea1ec4ea"
    "81d9e3fe50ea0f196d02a2f7cfe2c29c5577e35d8630baf6ea80465b01aa1abc"
    "394f57a1f4ccccda59ff8846e44b8805bb5cabe608c231f2dec8364ae7d90ab4"
    "358c5c3a421b06",
    # 6
    "6ef914ce5989152b321a769ad79c42c7be6f6ad2fab19de1fc339d84f04ad3a5"
    "89dfa0ff09ab0c196f13e7e780b4c097556ded57c871fbeea393464a01aa0ab1"
    "381848cfd2d6898918efc046b00b8940bb08e3f313cb23b3dfd8645cfcd80ff8"
    "2489",
    # 7
    "71fe1ace4389087266117cd7c4865bd2b93b7fd2b5a58ce9f4308c9ff01e97ab"
    "82cbf2ef5dfc101d6a56b3fb8ab4d08b4167ef5c9c30b8f0ab97455b45e81efd"
    "364605e49ddb83df48eedc42b60c900fb14db4b229ca74b6c4d96442e1c34df8"
    "288f5c3a450a527ecc7c82865b8e",
    # 8
    "71fe029a148c1437615978d7c58854dbec2c75cde5a39be5e37e9b97ef0697a2"
    "85dfa0f01cff101d764983f29bf5",
    # 9
    "71fe1ace50875b31730d6ad7cb8640c7ec3c73d4e1bf81e7b13796d6e518d8a4"
    "988ceff05dff101d2415a8fe9fe1d79a4623eb5e8430bfe3b3d442514faf40fd"
    "18420be0c8cb89924cf3cd5ee448950efd5cabe500c120f2d9d26440ebc34de0"
    "29811977430b01748276d79012955cc6a65aebb9054becda5c9278",
    # 10
    "71fe029a1483123c76597691878459cca9363ac4faf68ceffc2e8d82e61897b9"
    "8fc5e5f809e20b0c7756b2e08aab83bc5560e257",
```

```
]
TARGET_HEX = (
    "71fe0680149d083b7c1e3996879a42d0a92e7780f6bf9fe8f42cd898e61cd2b8"
    "ccd9f3f35dff101d241da2eacff9cc8d5123fe5a897efbeda4974b"
)

ciphertexts = [bytes.fromhex(h) for h in CIPHERTEXTS_HEX]
target = bytes.fromhex(TARGET_HEX)
all_cts = ciphertexts + [target]
names = [f"c{i}" for i in range(1, 11)] + ["T"]

print("lengths:", [len(c) for c in ciphertexts])
print("target :", len(target))
```

```
lengths: [78, 113, 90, 90, 103, 98, 110, 46, 123, 52]
target : 59
```

Step 1: Look before guessing

First 6 bytes of every ciphertext. The same key byte sits under each column.

In [15]:
```
for name, c in zip(names, all_cts):
    print(f"{name:<4}{c[:6].hex(' ')}")
```

```
c1  71 fe 1a ce 43 89
c2  71 fe 1a ce 55 9a
c3  72 fe 06 9c 51 c8
c4  67 e5 43 88 5b 9a
c5  71 fe 02 9a 14 8c
c6  6e f9 14 ce 59 89
c7  71 fe 1a ce 43 89
c8  71 fe 02 9a 14 8c
c9  71 fe 1a ce 50 87
c10 71 fe 02 9a 14 83
T   71 fe 06 80 14 9d
```

Eight of the eleven start with 71 fe, so those eight messages begin with the same two characters. c3 differs from them only in the first byte. c1 and c7 agree for even longer:

In [16]:
```
print(xor_bytes(ciphertexts[0], ciphertexts[6])[:20].hex(" "))
```

```
00 00 00 00 00 00 00 00 00 00 00 00 0d 0a 1a 50 3c 20 45 1d
```

Twelve zero bytes, so messages 1 and 7 start with the same 12 characters. I can see this without knowing anything about the key.

Step 2: The space trick

From Problem 3, a letter in ci XOR cj means one of the two has a space. For each position I look for the ciphertext that gives a letter (or a zero, when both have the same character) with the most other ciphertexts. That one probably has a space, so the key byte is that ciphertext byte XOR 0x20. I only accept a column if at least half of the other ciphertexts agree.

In [17]:
```
def guess_key_from_spaces(cts):
    """Guess key bytes from likely spaces. Positions without enough evidence stay None."""
    length = max(len(c) for c in cts)
    key = [None] * length
    for p in range(length):
        column = [c[p] for c in cts if len(c) > p]
        best_votes, best_byte = 0, None
```

```
        for i, ci in enumerate(column):
            # vote: this ciphertext XOR another one gives a letter, or zero
            votes = sum(1 for j, cj in enumerate(column)
                        if j != i and (is_letter(ci ^ cj) or ci == cj))
            if votes > best_votes:
                best_votes, best_byte = votes, ci ^ 0x20
        if best_votes >= max(2, (len(column) - 1) // 2):
            key[p] = best_byte
    return key


def decrypt_partial(c, key):
    """Decrypt with a partly known key ('_' = unknown key byte, '#' = not printable)."""
    out = ""
    for p, b in enumerate(c):
        if key[p] is None:
            out += "_"
        else:
            ch = b ^ key[p]
            out += chr(ch) if 32 <= ch < 127 else "#"
    return out


def show_all(key, start=0):
    """Print every ciphertext decrypted with the current key, from position `start` on."""
    for name, c in zip(names, all_cts):
        print(f"{name:<4}{decrypt_partial(c, key)[start:]}")
```

In [18]:

```
auto_key = guess_key_from_spaces(all_cts)
print("key bytes guessed:", sum(k is not None for k in auto_key), "of", len(auto_key))
print()
show_all(auto_key)
```

```
key bytes guessed: 88 of 123

c1    y was the n__ IT off__ial hospitalize_? H_ ac_id__tal_y _oucwe_ the firzwall
c2    y are the i__urance __d premiums for _ll _pp _ev__ope_s _norro_sly high  Because they f_e
__wa__ c__s_i_g d____
c3  # ere are 10 __pes of __ople in the wor_d. _hos_ w__ un_er_tan{ _inary an{ those who don _
c4  6; for punish__nt, whe__ are naughty di_k d_ive_ s__t? _he_ arz _lways seqt to a Boot caj_
c5    at did the __mputer __ on its much-aw_ite_ va_at__n a_ t_e bza_h? It ha{ a great time
t_rf__g __e __t
c6  ?'w many comp__er prog__mmers does it t_ke _o c_an__ a _ig_t bjl_? None; khat's a hardwau_
p__bl__
c7    y was the c__puter s__entist angry wh_n t_e s_ud__t c_ac_ed ~ _ame compjter joke? He dn_ n__
l__e __ _n_ bi_
c8    at does a b__y compu__r call his fath_r?D_ta
c9    y do cats l__e sitti__ in front of th_ co_put_r __l d_y _ong  _ecause twey don't want s_ l__
t__ m__s_ _ut _____________
c10   at kind of __ney do __mputer scientis_s u_e? _ac__
T     en using a __ream ci__er never use th_ ke_ mo_e __an _nc_
```

A lot is readable already, and the messages look like computer jokes. The target reads

```
en using a __ream ci__er never use th_ ke_ mo_e __an _nc_
```

The first two characters are wrong in every message. Almost all messages start with the same two letters, so the zero XORs counted as votes for a space. Most of the other gaps are columns where no message has a space, so the trick has nothing to work with there.

Step 3: Crib dragging

Before guessing the whole target, I tested a single word. The idea: assume the word sits at some position of the target, compute the key bytes that would give it, and check whether the other ten messages turn into text at the same spot. Checking against all ten at once gives every position a score, instead of a yes or no from one pair.

In [19]:
```
ALLOWED = set(b"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 ,.?")


def crib_drag(ct, others, word, top=5):
    """Try word at each position of ct, count how many other messages turn into text."""
    w = word.encode("utf-8")
    results = []
    for p in range(len(ct) - len(w) + 1):
        k = xor_bytes(ct[p:p + len(w)], w)
        pieces = [xor_bytes(o[p:p + len(w)], k) for o in others if len(o) >= p + len(w)]
        score = sum(all(b in ALLOWED for b in piece) for piece in pieces)
        results.append((score / max(len(pieces), 1), score, len(pieces), p, pieces))
    results.sort(key=lambda r: -r[0])
    for _, score, n, p, pieces in results[:top]:
        print(f"pos {p:>2}  {score}/{n}  {[x.decode('latin-1') for x in pieces]}")


crib_drag(target, ciphertexts, " the ")
```

```
pos 36  9/10  ['ized?', 'or al', 'world', ' disk', '-awai', 't tak', ' when', 'ather', ' the ',
'tists']
pos 12  8/10  ['nbk7\x0c', 'iiob7', ' seg ', 'hjyy1', ' dsz5', 'prhr7', 'chqg0', 'bf~ne', 'lhjre',
' jsy ']
pos 44  5/9  [' xdt,', "pi's ", 'hvtre', 'iobde', ' oft$', 'o9d\x7f$', 'e9tc0', 'plsr7', "e&'T$"]
pos 40  5/10  ['??Ey ', 'ls-}p', 'd1-Hh', 'k?ini', 'ikhx ', 'kz-ho', 'n?yte', 'r I}t', ' |bqp',
's?xoe']
pos 49  4/8  ['denp/', 'velk>', 'who$;', 'senpq', 'tiojn', 'nge$/', 'denpn', ' alhn']
```

Position 36 comes out on top with 9 of 10, and the pieces there are real word fragments (`ized?`, `world`, `disk`, `when`, ...). It also matches `use th_ ke_` in the target. Position 12 is close with 8 of 10, but its pieces are junk like `iiob7`, `hjyy1` and `chqg0`, which only pass because my filter accepts any letter or digit. So the score narrows it down, but I still had to read the pieces to pick the right position.

The one miss at position 36 is `-awai`, which has a hyphen. The handout says only space, comma, period and question mark are used, so my filter rejected it, but it is clearly part of "much-awaited".

Step 4: Guessing the whole target

The partial target `en using a __ream ci__er never use th_ ke_ mo_e __an _nc_` points to the well-known warning "When using a stream cipher never use the key more than once". That sentence is 59 characters and the target is 59 bytes. If the guess is right, the key bytes it gives must also turn the first 59 characters of all ten other messages into clean English.

In [20]:
```
def apply_crib(key, ct, position, guess):
    """Set key bytes so that ct decrypts to guess at this position (k = c XOR m)."""
    for offset, ch in enumerate(guess.encode("utf-8")):
        key[position + offset] = ct[position + offset] ^ ch
```

```
key = list(auto_key)
apply_crib(key, target, 0, "When using a stream cipher never use the key more than once")
show_all(key)
```

```
c1  Why was the new IT official hospitalized? He accidentally toucwe_ the firzwall
c2  Why are the insurance and premiums for all app developers enorro_sly high  Because they f_e
__wa__ c__s_i_g d____
c3  There are 10 types of people in the world. Those who understan{ _inary an{ those who don _
c4  As for punishment, where are naughty disk drives sent? They arz _lways seqt to a Boot caj_
c5  What did the computer do on its much-awaited vacation at the bza_h? It ha{ a great time
t_rf__g __e __t
c6  How many computer programmers does it take to change a light bjl_? None; khat's a hardwau_
p__bl__
c7  Why was the computer scientist angry when the student cracked ~ _ame compjter joke? He dn_ n__
l__e __ _n_ bi_
c8  What does a baby computer call his father?Data
c9  Why do cats love sitting in front of the computer all day long  _ecause twey don't want s_ l__
t__ m__s_ _ut _____________
c10 What kind of money do computer scientists use? Cache
T   When using a stream cipher never use the key more than once
```

All ten messages now read as normal English up to position 58, including the parts that were blank or wrong before (the "Wh" at the start, `hospitalized?`, `disk drives`, `much-awaited vacation`, `love sitting`). One guess fixed all 59 key bytes of the target, and ten other messages confirm it. So the target is decrypted.

Step 5: The rest of the key

Not needed for the target, but finishing the other messages is a good extra check. Past position 58, the partial text from Step 2 plus the context of each joke is enough to guess the full message. I did a few at a time and looked at the output after each group.

In [21]:
```
apply_crib(key, ciphertexts[0], 0, "Why was the new IT official hospitalized? He accidentally
touched the firewall")
apply_crib(key, ciphertexts[2], 0, "There are 10 types of people in the world. Those who
understand binary and those who don't")
apply_crib(key, ciphertexts[5], 0, "How many computer programmers does it take to change a light
bulb? None; that's a hardware problem")
show_all(key, start=55)
```

```
c1  ly touched the firewall
c2  rs enormously high? Because they are always c__s_i_g d____
c3  derstand binary and those who don't
c4  They are always sent to a Boot camp
c5  t the beach? It had a great time surfing the __t
c6  light bulb? None; that's a hardware problem
c7  racked a lame computer joke? He did not like __ _n_ bi_
c8
c9  ay long? Because they don't want to let the m__s_ _ut _____________
c10
T   once
```

These fit, and they also extended messages 2, 5, 7 and 9. Next, messages 5 and 7:

In [22]:
```
apply_crib(key, ciphertexts[4], 0, "What did the computer do on its much-awaited vacation at the
beach? It had a great time surfing the net")
apply_crib(key, ciphertexts[6], 0, "Why was the computer scientist angry when the student cracked
a lame computer joke? He did not like it one bit")
```

```
show_all(key, start=55)
```

```
c1  ly touched the firewall
c2  rs enormously high? Because they are always crashing do___
c3  derstand binary and those who don't
c4  They are always sent to a Boot camp
c5  t the beach? It had a great time surfing the net
c6  light bulb? None; that's a hardware problem
c7  racked a lame computer joke? He did not like it one bit
c8
c9  ay long? Because they don't want to let the mouse out o____________
c10
T   once
```

Only messages 2 and 9 are longer than 110 bytes, and only message 9 goes past 113. It reads "...let the mouse out o", so it must end with "out of their sight". Message 2 then gets its last three characters for free, which is a check on this guess.

In [23]:
```
apply_crib(key, ciphertexts[8], 0, "Why do cats love sitting in front of the computer all day
long? Because they don't want to let the mouse out of their sight")
show_all(key, start=55)
```

```
c1  ly touched the firewall
c2  rs enormously high? Because they are always crashing down!
c3  derstand binary and those who don't
c4  They are always sent to a Boot camp
c5  t the beach? It had a great time surfing the net
c6  light bulb? None; that's a hardware problem
c7  racked a lame computer joke? He did not like it one bit
c8
c9  ay long? Because they don't want to let the mouse out of their sight
c10
T   once
```

Message 2 ends with "crashing down!", which makes sense, so the guess for message 9 holds up at positions 110 to

112. The last 10 bytes (positions 113 to 122) only exist in message 9, so nothing else can confirm them.

Step 6: Final result

In [24]:
```
assert None not in key, "some key bytes are still unknown"
final_key = bytes(key)

print("TARGET:", xor_bytes(target, final_key).decode("utf-8"))
print()
for i, c in enumerate(ciphertexts, 1):
    print(f"{i:>2}: {xor_bytes(c, final_key).decode('utf-8')}")
print()
print("key:", final_key.hex())
```

```
TARGET: When using a stream cipher never use the key more than once

 1: Why was the new IT official hospitalized? He accidentally touched the firewall
 2: Why are the insurance and premiums for all app developers enormously high? Because they are
always crashing down!
 3: There are 10 types of people in the world. Those who understand binary and those who don't
 4: As for punishment, where are naughty disk drives sent? They are always sent to a Boot camp
 5: What did the computer do on its much-awaited vacation at the beach? It had a great time
surfing the net
```

```
 6: How many computer programmers does it take to change a light bulb? None; that's a hardware
problem
 7: Why was the computer scientist angry when the student cracked a lame computer joke? He did not
like it one bit
 8: What does a baby computer call his father?Data
 9: Why do cats love sitting in front of the computer all day long? Because they don't want to let
the mouse out of their sight
10: What kind of money do computer scientists use? Cache

key:
269663ee34e87b52127919f7a7e936a2cc4f1aa095d6ef80915ef8f6836ab7caecac80967d8b78780476c793ef94a3ff34
038a32e810db82caf42e3e21c87fdd5a276881bdb8ecb2389ba827c42cfa60da288b9261af54d2adbd442c8eb76d9441e4
391a2c7e7211a219a2e432fa3ae6d2328ed0776b9fb33bfa0c
```

Step 7: How much did the automatic part do?

In [25]:
```python
def compare(auto, final, positions):
    """Count positions where the space trick was right, wrong, or left empty."""
    positions = list(positions)
    right = sum(auto[p] == final[p] for p in positions)
    empty = sum(auto[p] is None for p in positions)
    return right, len(positions) - right - empty, empty


on_target = compare(auto_key, final_key, range(len(target)))
on_all = compare(auto_key, final_key, range(len(final_key)))
print("target (0-58): right %d, wrong %d, empty %d" % on_target)
print("whole key    : right %d, wrong %d, empty %d" % on_all)

plain = [xor_bytes(c, final_key) for c in all_cts]
missed = [p for p in range(len(final_key)) if auto_key[p] != final_key[p]]
no_space = [p for p in missed if all(len(m) <= p or m[p] != 0x20 for m in plain)]
print()
print(len(missed), "positions wrong or empty")
print(len(no_space), "of them have no space in any message")
print("wrong guesses at positions:", [p for p in missed if auto_key[p] is not None])
```

```
target (0-58): right 46, wrong 2, empty 11
whole key    : right 83, wrong 5, empty 35

40 positions wrong or empty
38 of them have no space in any message
wrong guesses at positions: [0, 1, 62, 73, 88]
```

Every column that went wrong or stayed empty has no space in any message, except two positions near the end where only one or two ciphertexts are left. Besides positions 0 and 1, the wrong guesses (62, 73, 88) are columns with a ? or an apostrophe in them, which can fake a space the same way the period did in Problem 3.

In [26]:
```python
for i, m in enumerate(plain[:10], 1):
    extra = sorted(chr(b) for b in set(m) - ALLOWED)
    if extra:
        print(f"message {i}: {extra}")
```

```
message 2: ['!']
message 3: ["'"]
message 5: ['-']
message 6: ["'", ';']
message 9: ["'"]
```

So the plaintexts do use a few characters the handout did not mention (! ' - ;). It did not stop the attack, but a strict character filter would have thrown out correct guesses, like `-awai` in Step 3.